SQL Injection (SQLi)

Week 6 — Web Security & Vulnerability Testing (VAPT Basics)

Technical Learning Notes | Prepared from authorized local lab work and OWASP guidance

1. Definition and overview

SQL Injection (SQLi) is a web application vulnerability that occurs when an application builds a database query by combining SQL commands with untrusted user input. If input is treated as part of the SQL statement rather than as data, an attacker may change the intended meaning of the query. The result depends on the query design, database permissions, application behavior, and deployment environment.

SQL injection is a vulnerability class, not a single payload. It can affect login forms, search fields, product lookups, URL parameters, API requests, and other inputs that reach database queries. Testing must be performed only on systems for which permission has been granted.

2. How it works

Consider an unsafe query constructed by string concatenation:

```
SELECT first_name, surname FROM users WHERE user_id = ' + user_input + ';
```

If the application inserts user input directly into the SQL text, special characters and SQL syntax may alter the query. The safe design is to keep the SQL statement structure separate from parameter values.

A parameterized example (illustrative pseudocode):

```
SELECT first_name, surname FROM users WHERE user_id = ?;
```

The application supplies the user ID as a parameter through the database driver. The database treats that value as data rather than executable SQL syntax. Parameter placeholder syntax varies by database and driver.

3. Common forms

- In-band SQLi: the same application channel returns database-related results or errors. Union-based and error-based techniques are examples.
- Blind SQLi: the application does not directly return the query result, so behavior differences are used to infer whether input affected the query. Boolean-based and time-based approaches are examples.
- Second-order SQLi: input is stored first and later becomes dangerous when another operation uses it unsafely in a query.

These categories describe techniques and behavior. A test result should be supported by observable evidence and should not be inferred from a scanner alert alone.

4. Impact and risk

Depending on the query and database account privileges, SQLi may allow unauthorized reading, modification, or deletion of records; bypass of application logic; exposure of personal or business information; and, in some configurations, further compromise. The practical impact is context-dependent. Risk assessment should consider data sensitivity, exploitability, required privileges, exposure, and the actual permissions available to the application database account.

5. Evidence from my authorized DVWA lab

In the local, intentionally vulnerable DVWA lab, the normal input `1` returned the record for `admin` (`First name: admin`, `Surname: admin`). The test input `1' OR '1'='1' #` returned multiple records. This behavior is evidence that the SQL Injection exercise in the configured DVWA lab was vulnerable at the time of testing.

Burp Suite was also used to inspect the local request to `/dvwa/vulnerabilities/sqli/` and replay the test in Repeater. With the lab configured to Low security, the response returned multiple records. This supports the lab finding. It does not establish that any production application is vulnerable.

Evidence to reference in the project:
• `02-Screenshots/01-SQL-Injection/` — SQLi normal-input and injection-test screenshots (use the actual saved filenames).
• `02-Screenshots/04-Burp-Suite/` — Burp request/response and Repeater evidence (use the actual saved filenames).
If a screenshot is not present in the final folder, do not claim it is included.

6. Prevention and secure design

1. Use prepared statements / parameterized queries for all values. Avoid building SQL with string concatenation.
2. Where dynamic identifiers (such as a sort column) are required, map user choices to a strict allowlist; identifiers often cannot be safely parameterized like ordinary values.
3. Validate input against expected type, length, and business rules. Validation is additional defense, not a substitute for parameterization.
4. Apply least privilege to database accounts. The web application should not connect as a database administrator and should have only the permissions it needs.
5. Do not expose detailed database errors to users. Log diagnostic details securely on the server.
6. Keep database libraries and application frameworks updated, and include SQLi-focused tests in code review and security testing.
7. Use monitoring and alerting for suspicious query errors or unusual access patterns, without logging secrets or sensitive data unnecessarily.

7. Verification and retesting

After remediation, repeat the same authorized test case and verify that the input is handled as a literal value, that unrelated records are not returned, and that normal functionality still works. Review server-side code to confirm parameterization; a single negative payload test alone is not proof that all injection paths are fixed. Record the test date, environment, input, observed result, and whether the fix was verified.

8. Key takeaway

SQL Injection occurs when untrusted input can change SQL query structure. Parameterized queries, least-privilege database access, safe error handling, and evidence-based retesting are core defenses. In this internship lab, the DVWA and Burp Suite results demonstrated SQLi behavior in the intentionally vulnerable local target.

References for further reading

- OWASP SQL Injection Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Web Security Testing Guide — https://owasp.org/www-project-web-security-testing-guide/
- OWASP Top 10 — https://owasp.org/www-project-top-ten/